Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Web-Based Paper Trading System for Stock Market Learning

The domain of the Project:

FinTech Combines with Web Development

COURSE NAME: G30 FullStack Development

Period of the project

March 2026 to April 2026

Declaration

The project titled Web-Based Paper Trading System for Stock Market Learning has been mentored by Akshara A, organised by SURE Trust, from March 2026 to April 2026, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Abstract

Recently, people have shown greater involvement in the financial markets as they now have easier access to them through digitization and more online trading platforms than ever before. However, an important issue exists despite the progress – numerous people, especially novices, still do not have enough experience and knowledge for proper trading operations. Even though a person can easily learn the theory from different literature, courses, and sources available on the internet, the process of translating theoretical knowledge into practice is far more complicated.

First, it is essential to understand that the market itself is extremely complex. Stocks and cryptocurrencies trading include numerous price fluctuations, affecting aspects, various assets and orders that people can work with. Novices are often not prepared for such complexity, which negatively impacts their ability to understand market dynamics, conduct necessary research, and apply all this knowledge to practice.

The problem is further accentuated by the fact that there is no provision of a safe and realistic environment in which they can practice. All real-world trading websites force users to use their money when trading, making them vulnerable financially. It makes them stressed and can hamper their decision-making process. They fear losses, get overconfident, and indulge in rash actions, which is quite common among novices, who cannot practice in a safe environment.

Additionally, there is an obvious discrepancy between education and practice. While educational material focuses on concepts related to technical indicators, basics of the market, and various trading strategies, they fail to provide practical insight into decision-making processes. This results in users being unable to convert what they have learned into practice and experience.

The usability and design of the interfaces are also factors behind the issue at hand. The current platforms available are either too complicated for newcomers or too simple to realistically simulate the workings of a trading floor. An interface overloaded with data, poor navigation, and technical language may confuse users and lead to disengagement. Similarly, overly simplified systems do not provide enough depth for effective learning. Such an imbalance in terms of complexity creates trouble for users who would like to find an interface appropriate for their needs.

Moreover, the divide between the learning materials and trading platforms is another factor affecting users. When studying, individuals need to alternate between tutorials or video content and actual trading sites. In such cases, the learning process becomes disjointed, as users cannot seamlessly integrate what they learn with practice.

Introduction

With the development of technology and its increasing accessibility through the internet, trading in the stock market has become easier for individuals than ever before. Through online trading systems, people have found a platform to conduct their trades without any geographical or intermediary limitations. Furthermore, it offers the advantage of providing real-time information related to the stock market, which means that users can make an instant decision regarding whether to buy or sell their stocks. Thus, many people, especially students and young professionals, have become attracted towards the world of investments. However, in spite of its easy accessibility, one major problem still remains. It is a well-known fact that the stock market is a complicated arena, as it includes dynamic price changes, volatility, and many more things. This makes it difficult for new entrants to understand these aspects and make wise decisions based on them. Moreover, although there are many educational materials available in the form of tutorials or books, they all tend to offer only a theoretical explanation of this area.

A considerable challenge that newbies have to face is the lack of a safe zone where they can experiment with trading without bearing any costs. The traditional method of trading on the stock market demands that users put money into their account, meaning that from the get-go, they face the possibility of losing some funds, especially at the beginning when their trading skills are not yet developed. Consequently, fear, hesitation, or impulsive actions might be induced by the fear of financial loss, resulting in either a loss or inactivity. Another problem associated with learning about trading in the stock market is the gap between the theoretical and practical parts of the process. For instance, traders need to know how to conduct technical and fundamental analyses, what types of orders to place, and how to manage their risks. However, without being able to practice trading, students cannot turn their theoretical knowledge into practical actions.

The next issue worth considering is the usability and design of current trading and simulation platforms. Most professional trading platforms have complicated interfaces, large data sets, and advanced analytical instruments. While all these elements are useful in terms of providing information, they can prove overwhelming for inexperienced users. Moreover, while some platforms strive to make the process of learning easier, they fail to provide an accurate depiction of the actual market activity. Such discrepancies emphasize the necessity of developing a tool that would be easy-to-use, user-friendly, and realistic enough.Finally, there is another problem associated with learning how to trade stocks. Namely, there is no correlation between the platform used to learn about the stock market and the one used for actual trading. As a result, users are forced to use two different applications, which negatively impacts the process of learning and applying the information received during training. Therefore, developing a platform where educational material could be supplemented by trading simulations would contribute to a more efficient learning experience.

Paper trading is an excellent idea to solve some problems associated with learning trading. The term "paper trading" means simulating trading operations by utilizing artificial money in the form of credits which can be used to buy and sell securities. In that way, paper trading gives the opportunity to gain practical experience in trading by implementing diverse techniques and testing one's skills under certain market conditions in a secure atmosphere. Paper trading does not involve real transactions, hence, there is no risk associated with monetary losses.

In this case, creating a website where people can perform paper trading turns out to be extremely beneficial. A web-based system simplifies everything as it does not require installation and is always accessible. Furthermore, such a platform can include useful features like real-time data visualization, interactive dashboards, and convenient user interface.

In addition, the implementation of web-based techniques would be helpful in enhancing the user experience level even further. Responsive design, real-time updates, and easy navigation can provide users with an interesting platform to interact with, which will be more convenient to use by a wide range of individuals. User-centered design is essential in developing an application that is comfortable for both beginners and advanced users.

The ability to track performance and get regular feedback is critical to the user learning process. Users need an opportunity to track their trades, analyze past decisions, and assess areas that require work. Without this important feature, users will have no way to assess their own progress and the impact of the actions made during the trading.

Scope

The scope of the project *"Web-Based Paper Trading System for Stock Market Learning"* is centered on providing a comprehensive, interactive, and risk-free environment for users to understand and practice stock market trading through a web-based platform. The system is designed to simulate real-world trading conditions while remaining accessible, user-friendly, and educationally focused. The following points define the specific scope and highlight the uniqueness of the project:

1. Web-Based Accessibility

The system is developed as a web application, enabling users to access the platform from any device with an internet connection without requiring installation. This ensures cross-platform compatibility and ease of use, making the system widely accessible to students and beginners.

2. Simulated Trading Environment

The project provides a virtual trading interface where users can perform buy and sell operations using simulated funds. The platform replicates essential trading functionalities

such as order placement, portfolio management, and transaction history, allowing users to experience real trading workflows without financial risk.

3. Learning-Oriented Design

Unlike conventional trading platforms, the system emphasizes learning by integrating educational components such as guided tutorials, video-based learning modules, and simplified explanations of trading concepts. This creates a structured environment that supports both theoretical understanding and practical application.

4. User-Friendly Interface

The platform is designed with a focus on simplicity and usability, ensuring that beginners can easily navigate through different features such as trading, research, and learning modules. The interface avoids unnecessary complexity while maintaining sufficient functionality to reflect realistic trading scenarios.

5. Integrated Research and Analysis Module

The system includes a research section where users can view stock-related information such as price trends, basic financial metrics, and market summaries. This integration allows users to analyze data and make informed decisions within the same platform, eliminating the need to switch between multiple tools.

6. Portfolio and Performance Tracking

Users can monitor their virtual investments through portfolio tracking features that display holdings, trade history, and account balance. This enables users to evaluate their trading decisions and understand performance over time, which is essential for skill development.

7. Modular System Design

The project is structured into distinct modules such as authentication (login/register), trading dashboard, research tab, and learning section. This modular approach enhances scalability, making it easier to extend the system with additional features such as real-time data integration or advanced analytics in the future.

8. Real-Time Simulation Capability (Extendable)

Although the system may initially use mock or delayed data, it is designed to support integration with real-time stock market APIs. This ensures that the platform can evolve into a more realistic simulation environment without major architectural changes.

9. Risk-Free Skill Development

A key aspect of the project's scope is enabling users to experiment with trading strategies without financial consequences. This reduces the psychological barrier associated with real trading and encourages continuous learning through trial and error.

10. Unique Integration of Learning + Trading

The most distinctive feature of this project is the seamless integration of learning resources with a trading simulator. Unlike traditional systems where education and execution are separate, this platform combines both aspects into a single environment, allowing users to directly apply concepts as they learn.

Existing System vs Proposed System

1. Existing System

The existing system for stock market learning and trading is primarily divided into educational platforms and real-time trading platforms. These systems operate independently and are not designed to provide a complete learning experience for beginners. While educational platforms focus on theoretical knowledge, trading platforms emphasize execution using real money. This separation creates a gap between understanding concepts and applying them in real-world scenarios.

In most cases, beginners start by learning from online tutorials or courses and then move to trading platforms. However, this transition is not smooth because the knowledge gained is not directly applied in a practical environment. As a result, users often face confusion, lack confidence, and make incorrect decisions.

Key Limitations of Existing System:

- **Separation of Learning and Practice:** Users must switch between different platforms.
- **Lack of Real-Time Practice:** No proper simulation environment for beginners.
- **High Entry Barrier:** Beginners find it difficult to start trading.
- **Limited Skill Development:** Practical skills are not developed effectively.
- **Inconsistent Learning Flow:** Disconnection between theory and execution.

2. Educational Platforms (Existing Subsystem)

Educational platforms provide foundational knowledge about stock markets through videos, articles, and tutorials. These platforms explain concepts such as stock trading basics, technical indicators, and investment strategies. Although they are useful for understanding theory, they do not provide an environment for applying these concepts in real-time.

Learning from such platforms is mostly passive, meaning users only consume information without interacting with a system that allows them to test their understanding. This results in limited retention and difficulty in applying knowledge during actual trading situations.

Limitations:

- **No Practical Application:** Users cannot execute trades within the platform.
- **Passive Learning Approach:** Lack of interaction reduces engagement.
- **No Real-Time Market Simulation:** Concepts are not tested in dynamic conditions.
- **No Feedback Mechanism:** Users cannot evaluate their understanding.
- **Dependence on External Platforms:** Requires switching to trading platforms for practice.

3. Online Trading Platforms (Existing Subsystem)

Online trading platforms provide users with the ability to trade financial instruments such as stocks and cryptocurrencies using real money. These platforms offer real-time data, charts, and advanced tools for analysis. However, they are primarily designed for experienced users and may not be suitable for beginners.

The presence of financial risk makes it difficult for new users to experiment and learn. Beginners often face losses due to lack of experience and understanding of market behavior. Additionally, the complexity of these platforms can overwhelm users, making it harder to navigate and utilize the available features effectively.

Limitations:

- **Financial Risk:** Users must invest real money.
- **Complex Interface:** Difficult for beginners to understand.
- **Overloaded Information:** Too many tools and indicators.
- **No Guided Learning:** Lack of structured educational support.
- **Psychological Pressure:** Fear of loss affects decision-making.

4. Problems in Existing System

The combination of educational and trading platforms fails to provide a comprehensive solution for beginners. The major problem lies in the lack of integration between learning and practice, which prevents users from gaining confidence and practical skills.

Beginners often struggle with understanding how to apply theoretical knowledge in real-time trading scenarios. Without a safe environment to experiment, users are hesitant to try different strategies. This results in slow learning and reduced participation in financial markets.

Major Problems Identified:

- **Gap between Theory and Practice**
- **Lack of Risk-Free Environment**
- **Poor User Experience for Beginners**
- **Fragmented Learning Process**
- **Limited Performance Evaluation Tools**

5. Proposed System

The proposed system, *"Web-Based Paper Trading System for Stock Market Learning,"* is designed to overcome the limitations of existing systems by providing a unified platform that integrates learning, research, and trading simulation. It offers a risk-free environment where users can practice trading using virtual money while gaining real-time experience.

The system focuses on providing an interactive and user-friendly interface that simplifies complex trading processes. By combining educational resources with a simulated trading environment, the platform ensures a continuous and effective learning experience.

Key Features of Proposed System:

- **Paper Trading Simulation:** Allows trading with virtual funds.
- **Integrated Learning Module:** Combines theory and practice.
- **User-Friendly Interface:** Designed specifically for beginners.
- **Trade Execution System:** Supports buy/sell operations.
- **Research Module:** Provides stock data and insights.
- **Portfolio Tracking:** Displays holdings and account balance.
- **Performance Monitoring:** Helps users analyze their trades.
- **Web-Based Platform:** Accessible from any device.

6. Unique Features of Proposed System

The uniqueness of the proposed system lies in its ability to combine multiple functionalities into a single platform. Unlike existing systems, it does not separate learning and trading but integrates them seamlessly.

This integration ensures that users can immediately apply what they learn, observe the outcomes, and improve their understanding. The system also reduces complexity by providing a simplified interface while maintaining essential trading features.

Unique Aspects:

- **Learning + Trading Integration**
- **Risk-Free Practice Environment**
- **Simplified User Experience**
- **Continuous Learning Workflow**
- **Beginner-Centric Design**

7. Improvements Over Existing System

The proposed system significantly enhances the overall learning experience by addressing the limitations of existing systems. It provides a structured and interactive environment where users can develop trading skills without financial risk.

By offering simulation, research, and learning features in one platform, the system ensures that users can progress from basic understanding to practical expertise in a smooth and efficient manner.

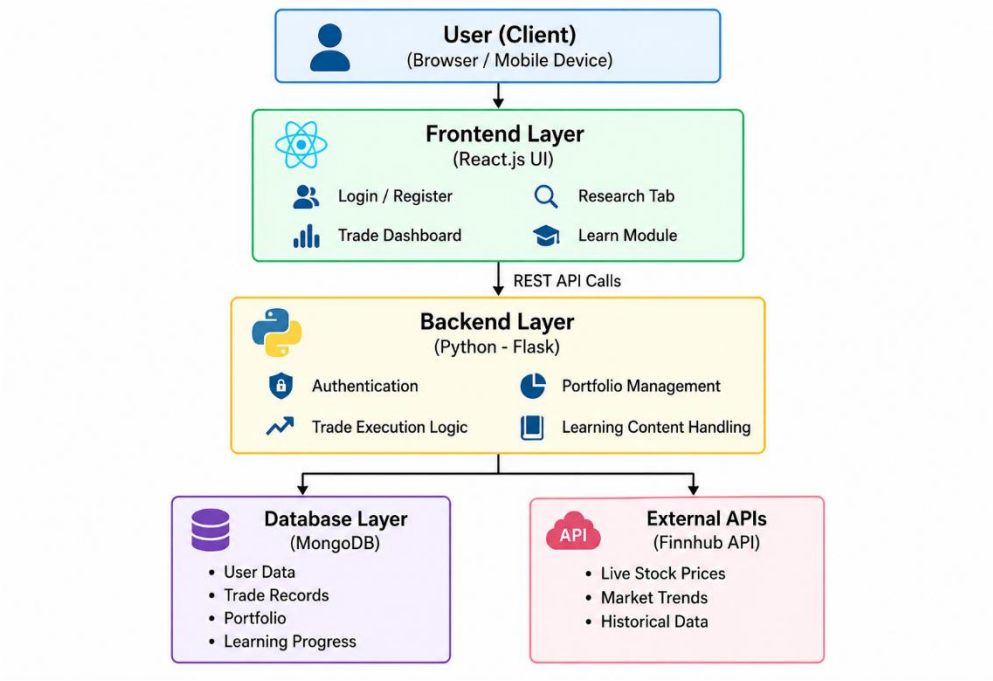
Major Improvements:

- **Eliminates Financial Risk**
- **Provides Hands-On Experience**
- **Improves User Engagement**
- **Enhances Learning Efficiency**
- **Supports Skill Development**
- **Offers Better Usability**

Architectural Diagram

The architecture of the *Web-Based Paper Trading System for Stock Market Learning* is designed using a layered (multi-tier) architecture, which separates the system into distinct functional layers: Client Layer, Frontend Layer, Backend Layer, Database Layer, and External API Layer. This architectural approach enables the system to be organized in a structured and modular manner, where each layer is responsible for a specific set of tasks and communicates with other layers through well-defined interfaces.

By isolating functionalities into separate layers, the system achieves high **scalability**, allowing new features such as advanced analytics, AI-based recommendations, or additional trading modules to be integrated without affecting the existing components.



1. Client Layer (User Layer)

The client layer represents the end users who access the system through a web browser or mobile device. It acts as the interface through which users interact with the application. Users can perform operations such as account registration, login, executing trades, viewing portfolios, analyzing stocks, and accessing learning materials. This layer is responsible only for sending user inputs and displaying outputs, without handling any business logic.

Functions:

- Sending requests to frontend
- Displaying processed results

2. Frontend Layer (Presentation Layer)

The frontend layer is developed using **React.js**, which provides a dynamic and responsive user interface. It acts as a bridge between the user and the backend system.

This layer handles all visual components and ensures smooth navigation across different modules such as trading, research, and learning. It communicates with the backend using REST API calls to send user requests and receive data.

Key Features:

- Interactive and responsive UI
- Real-time updates of stock information

Main Modules:

- Login / Registration
- Trade Dashboard
- Research Tab
- Learn Module

3. Backend Layer (Application Layer)

The backend layer is implemented using **Python (Flask)** and serves as the core processing unit of the system. It handles all business logic, data processing, and communication between the frontend, database, and external APIs.

This layer processes user requests, executes trading logic, manages portfolios, and ensures secure authentication. It also integrates with external APIs to fetch real-time stock market data.

Responsibilities:

- User authentication and authorization
- Trade execution (buy/sell simulation)
- Portfolio management and calculations
- API integration with external services

Core Components:

- Authentication module
- Portfolio management system
- Data processing logic

4. Database Layer (Data Storage Layer)

The database layer uses **MongoDB**, a NoSQL database, to store all application data. It ensures efficient storage and retrieval of structured and semi-structured data. This layer maintains all persistent data required by the system, enabling users to track their trading activities and progress over time.

Stored Data Includes:

- User account details
- Trade history records
- Portfolio information

5. External API Layer (Finnhub API)

The system integrates with the **Finnhub API**, which provides real-time and historical stock market data. This integration enhances the realism of the paper trading experience.

The backend fetches data from the API and sends it to the frontend for display and analysis.

Data Provided:

- Live stock prices
- Market trends
- Historical data for charts

6. Data Flow in the System

The system follows a structured data flow:

- The user performs an action (e.g., buy stock) through the frontend.
- The frontend sends a request to the backend using REST APIs.
- The backend processes the request and applies business logic.
- The backend interacts with the database to store or retrieve data.
- If required, the backend fetches real-time data from the Finnhub API.
- The backend sends the response back to the frontend.
- The frontend updates the UI and displays the result to the user.

Tech stack

1. Frontend Technologies

1.1 React.js

React.js is a JavaScript library used for building dynamic and interactive user interfaces. It allows developers to create reusable components and ensures efficient rendering using the virtual DOM. In this project, React.js is used to develop all major UI modules such as the login and registration pages, trading dashboard, research tab, and learning module. Its component-based structure helps in maintaining clean and scalable code while providing a smooth user experience through a single-page application approach.

Key Points:

- Component-based architecture enables code reusability
- Supports Single Page Application (SPA) for smooth navigation
- Efficient state management for handling trading data

1.2 JavaScript (ES6+)

JavaScript is used to add interactivity and dynamic behavior to the frontend. It enables communication with the backend and updates the UI in real time based on user actions and data changes.

Key Points:

- Handles user events (click, input, submit)
- Performs API calls to the backend
- Dynamically updates UI components
- Enables real-time interaction

2. Backend Technologies

2.1 Python

Python is used as the primary backend programming language due to its simplicity, readability, and strong support for web development. It efficiently handles data processing and server-side logic in the application.

Key Points:

- Easy to learn and implement
- Strong support for backend development
- Efficient for data processing and handling
- Widely used in modern web applications

2.2 Flask (Backend Framework)

Flask is a lightweight web framework used to build RESTful APIs. It acts as the core backend system that processes requests from the frontend and returns appropriate responses. Flask handles business logic such as trade execution and user authentication.

Key Points:

- Handles HTTP requests (GET, POST)
- Implements trade execution logic
- Connects frontend with database

- Lightweight and flexible framework
- Suitable for small to medium applications

3. Database Technologies

MongoDB

MongoDB is a NoSQL database used for storing application data in a flexible, JSON-like format. It allows efficient storage and retrieval of data related to users, trades, and portfolios.

Key Points:

- Stores user details and credentials
- Maintains trade history records
- Manages portfolio data
- Tracks learning progress
- Flexible schema design
- Easy integration with Python

4. External API Integration

Finnhub API

Finnhub API is used to fetch real-time and historical stock market data. It plays a crucial role in simulating real market conditions within the paper trading system.

Key Points:

- Provides live stock prices
- Supplies market trends and updates
- Offers historical data for analysis
- Easy to integrate with backend
- Supports both stock and cryptocurrency data

Code Snippets

The application is developed using a **component-based architecture**, which is a modern software design approach where the entire system is divided into smaller, independent, and reusable units called components. Each component is responsible for a specific functionality or part of the user interface, such as forms, buttons, dashboards, or data displays. These components can be developed, tested, and maintained independently, making the overall system more modular and organized.

- **Registeration Page**

```python
import React, { useState } from "react";
import {
  TrendingUp,
  Mail,
  Lock,
  User,
  Eye,
  EyeOff,
  ArrowRight,
  Loader2,
  CheckCircle2,
  Rocket,
  LineChart
} from "lucide-react";

export default function AuthPage({ onLoginSuccess }) {
  const [isActive, setIsActive] = useState(false);
  const [showPassword, setShowPassword] = useState({
    signIn: false,
    signUp: false,
  });
  const [isLoading, setIsLoading] = useState(false);
  const [isSuccess, setIsSuccess] = useState(false);

  const togglePassword = (type) => {
    setShowPassword((prev) => ({
      ...prev,
      [type]: !prev[type],
    }));
  };

  const handleSubmit = async (e, type) => {
    e.preventDefault();
    setIsLoading(true);

    const formData = new FormData(e.target);
    const email = formData.get("email");
    const password = formData.get("password");
```

```python
const handleSubmit = async (e, type) => {
  const username = formData.get("username") || email.split('@')[0];

  try {
    const endpoint = type === "signup" ? "register" : "login";
    const payload = type === "signup"
      ? { username, email, password }
      : { email, password };

    const response = await fetch(`http://localhost:8000/api/auth/${endpoint}`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });

    const data = await response.json();
    if (!response.ok) throw new Error(data.detail || "Authentication failed");

    setIsLoading(false);
    setIsSuccess(true);

    setTimeout(() => {
      setIsSuccess(false);
      if (onLoginSuccess) {

        onLoginSuccess(type === "signup" ? { name: username, email: email } : data.user);
      }
    }, 1000);
  } catch (err) {
    alert(err.message);
    setIsLoading(false);
  }
};

return (
  <div class="min-h-screen bg-slate-50 flex items-center justify-center p-4 relative overflow-hidden font-sans">
    {}
    <div class="absolute top-0 -left-4 w-72 h-72 bg-blue-100 rounded-full mix-blend-multiply filter blur-3xl opacity-40 animate-blob"></div>
```

```c
.container-wrapper {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  position: relative;
  overflow: hidden;
}

.bg-blobs {
  position: absolute;
  inset: 0;
  z-index: 0;
  pointer-events: none;
}

.blob {
  position: absolute;
  border-radius: 50%;
  filter: blur(100px);
  opacity: 0.5;
  animation: drift 18s ease-in-out infinite alternate;
}

.blob-1 {
  width: 600px; height: 600px;
  background: hsl(252, 82%, 63%);
  top: -20%; left: -10%;
}

.blob-2 {
  width: 500px; height: 500px;
  background: hsl(280, 80%, 50%);
  bottom: -15%; right: -5%;
  animation-delay: -6s;
}
```

```c
.sign-in-container { left: 0; z-index: 2; opacity: 1; }
.sign-up-container { left: 0; z-index: 1; opacity: 0; pointer-events: none; }

.container.active .sign-in-container {
  transform: translateX(100%);
  opacity: 0;
  z-index: 1;
  pointer-events: none;
}

.container.active .sign-up-container {
  transform: translateX(100%);
  opacity: 1;
  z-index: 5;
  pointer-events: all;
}

form {
  height: 100%;
  display: flex;
  flex-direction: column;
  justify-content: center;
  padding: 48px 44px;
  color: #f0f0f5;
}

form h1 {
  font-size: 1.8rem;
  font-weight: 700;
  margin-bottom: 4px;
  background: linear-gradient(135deg, #fff 30%, hsl(252, 82%, 63%));
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}

.subtitle {
  font-size: 0.85rem;
  color: rgba(255, 255, 255, 0.65);
```

- **Research Page**

```python
import React, { useState } from "react";
import "./Research.css";

export default function Research() {
  const [symbol, setSymbol] = useState("NVDA");

  const stock = {
    name: "NVIDIA CORPORATION",
    price: 208.27,
    change: "+8.63 (+4.32%)",
    market: "Nasdaq Stock Market",
    eps: 4.93,
    marketCap: "5.06T",
    pe: 40.73,
    high: 210.95,
    low: 199.81,
  };

  return (
    <div className="research-page">

      {}
      <div className="search-box">
        <label>Symbol</label>
        <div className="search-input">
          <input
            value={symbol}
            onChange={(e) => setSymbol(e.target.value)}
            placeholder="Search symbol..."
          />
          <span>🔍</span>
        </div>
      </div>

      {}
      <div className="research-card">

        {}
```

```python
<div className="stock-info">

  <h1>{symbol}</h1>
  <p className="company">{stock.name}</p>
  <p className="market">{stock.market}</p>

  <div className="price">
    ₹{stock.price} <span className="green">{stock.change}</span>
  </div>

  {}
  <div className="fundamentals">
    <div>
      <span>EPS</span>
      <p>{stock.eps}</p>
    </div>
    <div>
      <span>Market Cap</span>
      <p>{stock.marketCap}</p>
    </div>
    <div>
      <span>P/E</span>
      <p>{stock.pe}</p>
    </div>
  </div>

  {}
  <div className="stats">
    <div>
      <span>Day High</span>
      <p>{stock.high}</p>
    </div>
    <div>
      <span>Day Low</span>
      <p>{stock.low}</p>
    </div>
  </div>
```

```css
.research-page {
  background: #2f3b55;
  min-height: 100vh;
  padding: 20px;
  color: white;
}

.search-box {
  margin-bottom: 20px;
}

.search-input {
  display: flex;
  align-items: center;
  background: #e5e7eb;
  border-radius: 6px;
  padding: 10px;
}

.search-input input {
  flex: 1;
  border: none;
  outline: none;
  background: transparent;
}

.research-card {
  background: #f5f5f5;
  color: black;
  border-radius: 8px;
  padding: 20px;
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
}

.stock-info h1 {
  font-size: 40px;
```

```css
.market {
  font-size: 13px;
  margin-bottom: 10px;
}

.price {
  font-size: 30px;
  margin: 10px 0;
}

.green {
  color: green;
}

.fundamentals {
  display: flex;
  gap: 20px;
  margin-top: 10px;
}

.stats {
  display: flex;
  gap: 20px;
  margin-top: 10px;
}

.actions {
  margin-top: 20px;
  display: flex;
  gap: 10px;
}

.trade-btn {
  flex: 1;
  padding: 10px;
  border: 1px solid #3b82f6;
  background: transparent;
  color: #3b82f6;
}
```

- **Trade Page**

```
You, 21 minutes ago | 1 author (You)
import React, { useState, useEffect, useMemo } from "react";
import {
  TrendingUp,
  Search,
  ArrowRight,
  Info,
  ChevronDown,
  ShieldCheck,
  Zap
} from "lucide-react";

import TradingViewChart from "./TradingViewChart";

export default function TradePage({ user }) {
  const [action, setAction] = useState("BUY");
  const [orderType, setOrderType] = useState("MARKET");
  const [quantity, setQuantity] = useState("");
  const [price, setPrice] = useState("");
  const [symbol, setSymbol] = useState("");
  const [stockData, setStockData] = useState(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState(null);
  const [suggestions, setSuggestions] = useState([]);
  const [showSuggestions, setShowSuggestions] = useState(false);
  const [activeSubTab, setActiveSubTab] = useState("STOCKS");
  const [tradeHistory, setTradeHistory] = useState([]);

  const optionsData = useMemo(() => [
    { s: -10, itm: true }, { s: -5, itm: true }, { s: 0, itm: false },
    { s: 5, itm: false }, { s: 10, itm: false }, { s: 15, itm: false }
  ].map(row => ({
    ...row,
    callLast: (5 - row.s * 0.4 + Math.random()).toFixed(2),
    callChg: (Math.random() * 2 - 1).toFixed(2),
    callChgColor: Math.random() > 0.5 ? 'text-emerald-600' : 'text-red-600',
    callVol: Math.floor(Math.random() * 5000).toLocaleString(),
    callOI: Math.floor(Math.random() * 2000).toLocaleString(),
    putLast: (5 + row.s * 0.4 + Math.random()).toFixed(2),
```

```
const fetchTradeHistory = async () => {
  if (!user?.email) return;
  try {
    const response = await fetch(`http://localhost:8000/api/user/trades?email=${user.email}`);
    const data = await response.json();
    setTradeHistory(data.reverse());
  } catch (err) {
    console.error("Failed to fetch trades:", err);
  }
};

const fetchStockPrice = async (ticker) => {
  if (!ticker) return;
  setIsLoading(true);
  setError(null);
  try {
    const response = await fetch(`http://localhost:8000/api/stock/${ticker}`);
    if (!response.ok) throw new Error("Stock not found");
    const data = await response.json();
    setStockData(data);
    if (orderType === "MARKET") {
      setPrice(data.price.toString());
    }
  } catch (err) {
    setError(err.message);
    setStockData(null);
  } finally {
    setIsLoading(false);
  }
};

const [account, setAccount] = useState({
  totalAccountValue: 100000.0,
  buyingPower: 100000.0,
  cash: 100000.0,
});
```

```
.trade-page {
  background: #0f172a;
  min-height: 100vh;
  color: white;
  padding: 20px;
}

.trade-header {
  margin-bottom: 20px;
}

.account-stats {
  display: flex;
  gap: 20px;
  font-size: 14px;
  opacity: 0.8;
}

.market-open {
  color: #00ff88;
}

.trade-layout {
  display: grid;
  grid-template-columns: 2fr 1fr;
  gap: 20px;
}

.trade-form {
  background: #1e293b;
  padding: 20px;
  border-radius: 12px;
}

.field {
  margin-bottom: 15px;
}
```

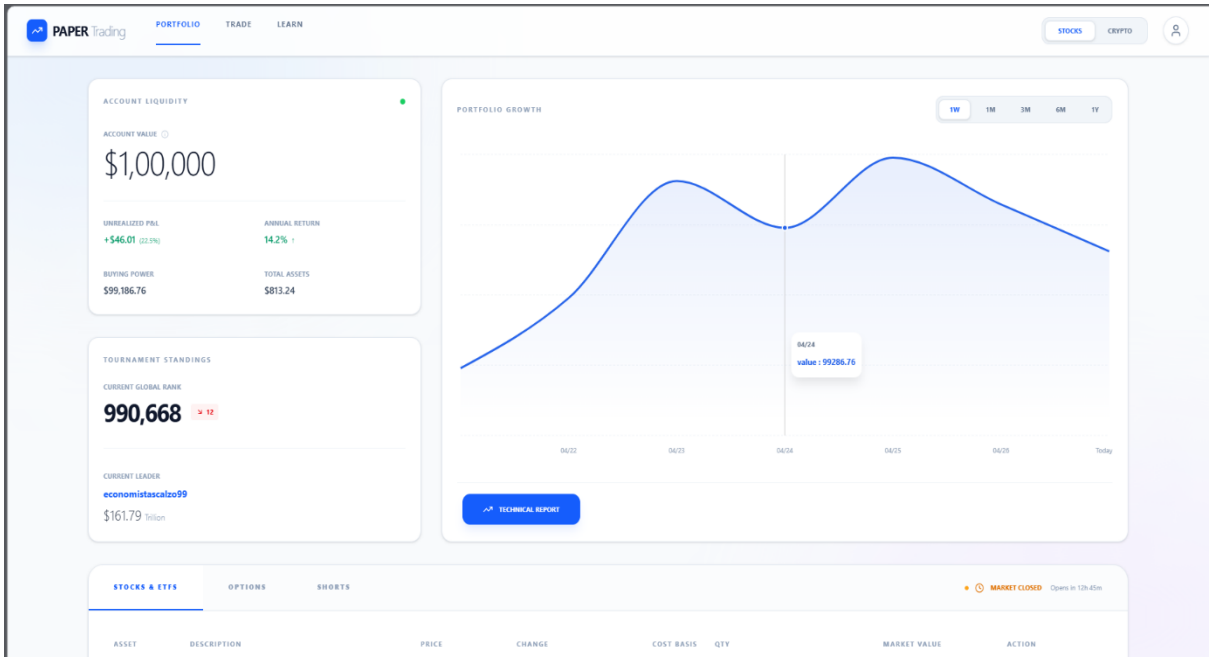
```
.field label {
  display: block;
  margin-bottom: 6px;
  font-size: 14px;
}

input {
  width: 100%;
  padding: 10px;
  border-radius: 8px;
  border: none;
  background: #0f172a;
  color: white;
}

.toggle {
  display: flex;
  gap: 10px;
}

.toggle button {
  flex: 1;
  padding: 10px;
  background: #0f172a;
  border: none;
  color: white;
  cursor: pointer;
}

.toggle .active {
  background: #6366f1;
}

.buy.active {
  background: #16a34a;
}

.sell.active {
  background: #dc2626;
```

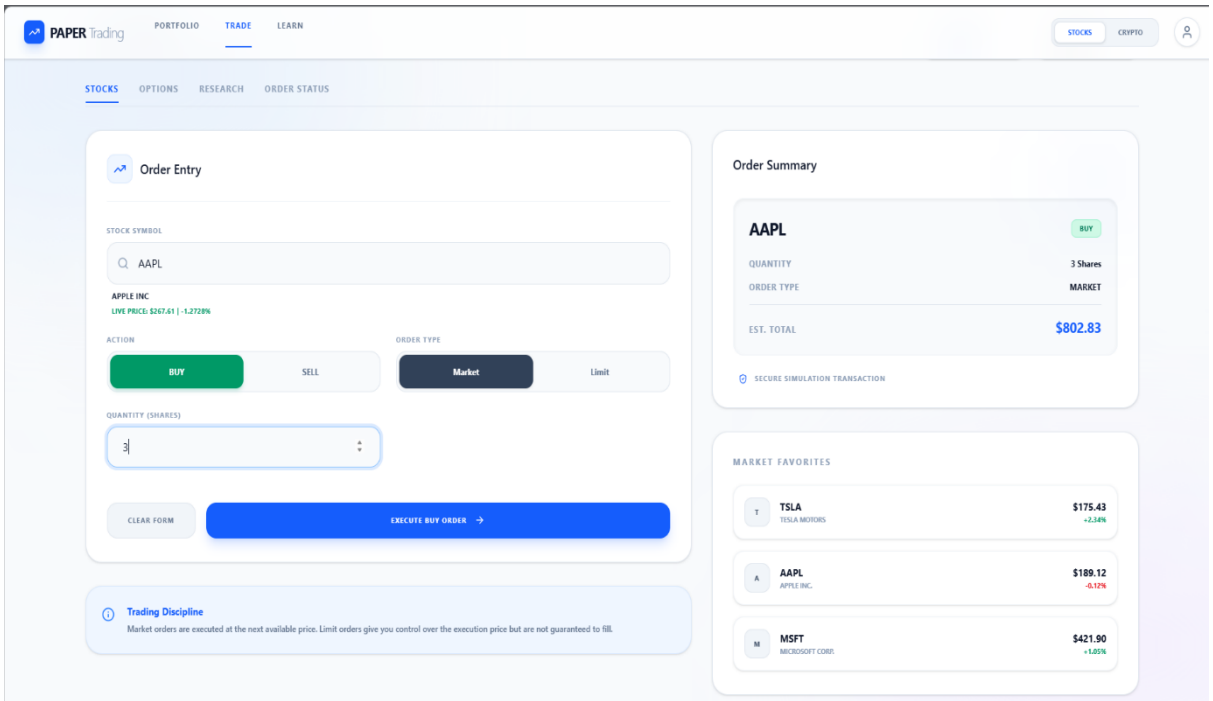
- **Dasboard**

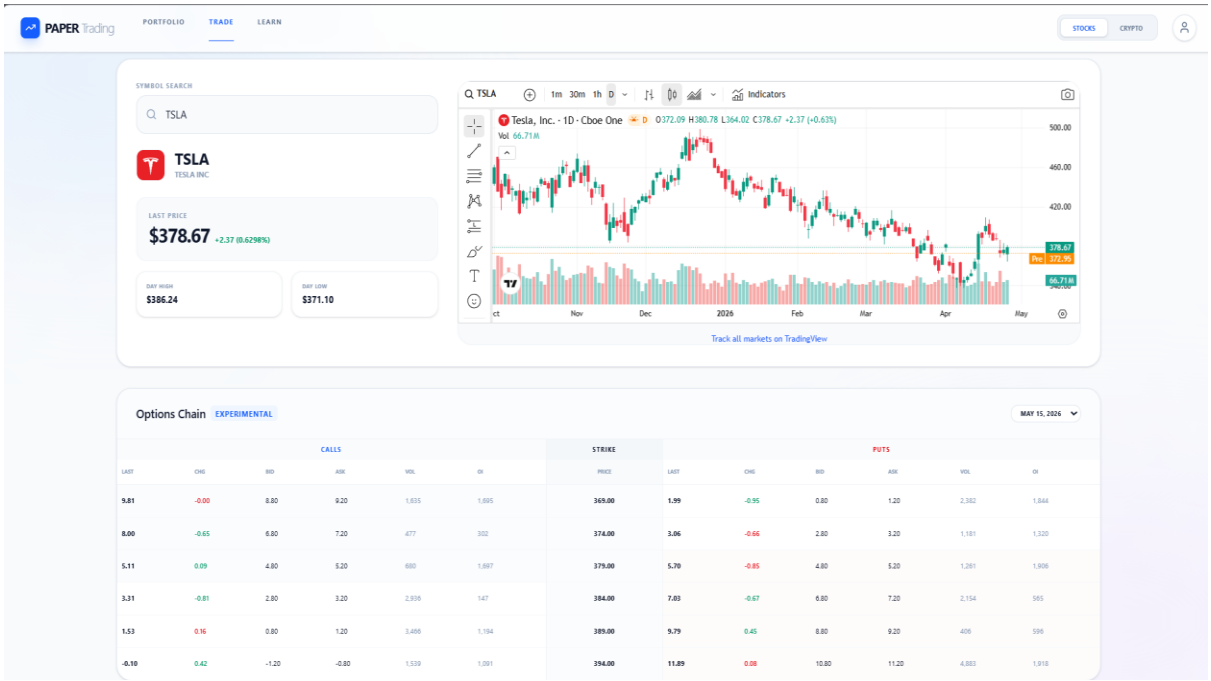
```
import React from 'react';
import {
  AreaChart,
  Area,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  ResponsiveContainer,
  TrendingUp,
  Info,
  User,
  ArrowDownRight,
  Clock
} from 'lucide-react';
import { AreaChart as RechartsAreaChart, Area as RechartsArea, XAxis as RechartsXAxis, YAxis as RechartsYAxis, CartesianGrid as RechartsCartesianGri

import TradePage from '../trade/TradePage';
import Learn from '../learn/Learn';

const Home = ({ user, onLogout }) => {
  const [activeTab, setActiveTab] = React.useState('portfolio');
  const [showUserMenu, setShowUserMenu] = React.useState(false);
  const [portfolioData, setPortfolioData] = React.useState(null);
  const [historyData, setHistoryData] = React.useState([]);

  React.useEffect(() => {
    const fetchData = async () => {
      if (!user?.email) return;
      try {
        const pRes = await fetch(`http://localhost:8000/api/user/portfolio?email=${user.email}`);
        const pData = await pRes.json();
        setPortfolioData(pData);

        const hRes = await fetch(`http://localhost:8000/api/user/portfolio/history?email=${user.email}`);
        const hData = await hRes.json();
        setHistoryData(hData);
```

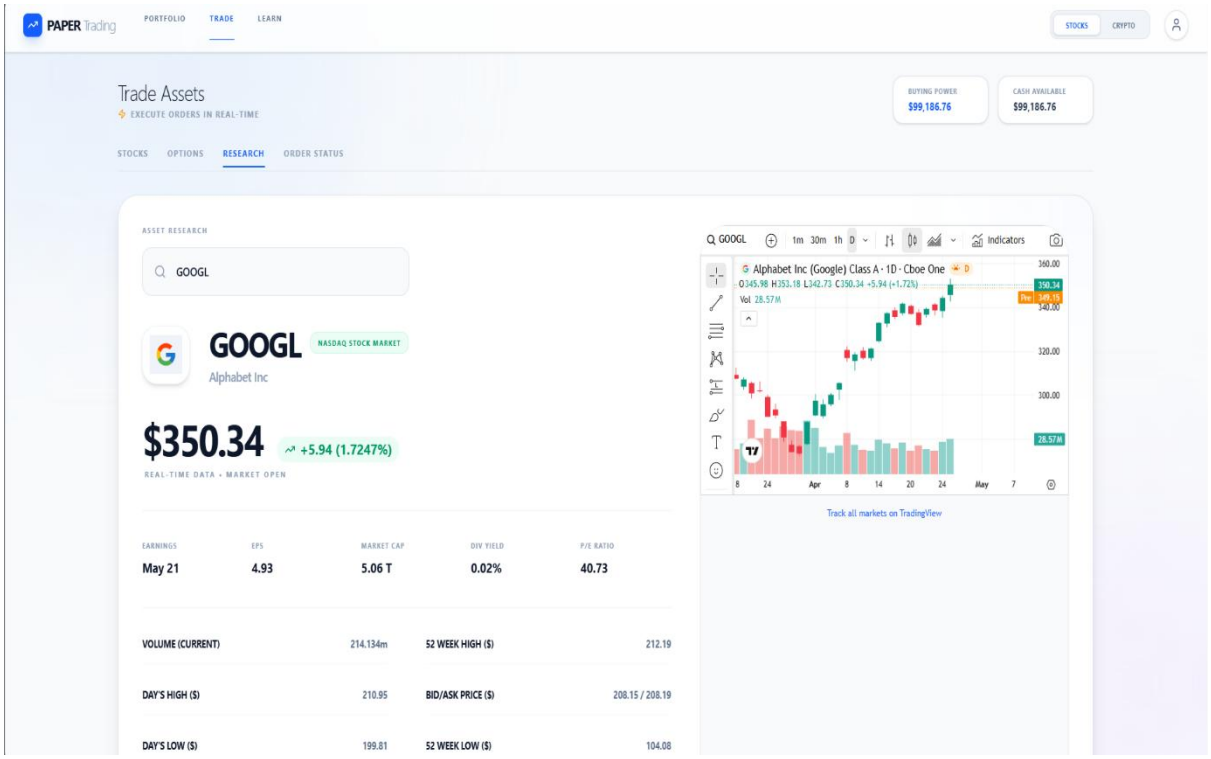
```
return (
  <div className="min-h-screen bg-slate-50 text-slate-900 font-sans selection:bg-blue-500/10">
    <div className="fixed inset-0 z-0 pointer-events-none overflow-hidden">
      <div className="absolute top-[-10%] left-[-10%] w-[40%] h-[40%] bg-blue-100/50 rounded-full blur-[120px]" />
      <div className="absolute bottom-[-10%] right-[-10%] w-[40%] h-[40%] bg-purple-100/50 rounded-full blur-[120px]" />
    </div>

    <nav className="relative z-20 bg-white/80 backdrop-blur-xl px-8 py-4 flex items-center justify-between border-b border-slate-200 sticky top-0 shadow-sm">
      <div className="flex items-center gap-12">
        <div className="text-xl font-bold tracking-tighter flex items-center gap-2">
          <div className="w-8 h-8 bg-blue-600 rounded-lg flex items-center justify-center shadow-lg shadow-blue-600/20">
            <TrendingUp size={18} className="text-white" />
          </div>
          <span className="text-slate-900">
            PAPER <span class="font-light text-slate-400">Trading</span>
          </span>
        </div>
        <div className="hidden lg:flex gap-10 text-[11px] font-bold uppercase tracking-widest text-slate-500">
          <span
            onClick={() => setActiveTab('portfolio')}
            className={`pb-5 translate-y-[2px] cursor-pointer transition-all duration-300 ${activeTab === 'portfolio' ? 'text-blue-600 border-b-2 border-blue-600' : 'hover:text-
          >
            Portfolio
          </span>
          <span
            onClick={() => setActiveTab('trade')}
            className={`pb-5 translate-y-[2px] cursor-pointer transition-all duration-300 ${activeTab === 'trade' ? 'text-blue-600 border-b-2 border-blue-600' : 'hover:text-slat
          >
            Trade
          </span>
          <span
            onClick={() => setActiveTab('learn')}
            className={`pb-5 translate-y-[2px] cursor-pointer transition-all duration-300 ${activeTab === 'learn' ? 'text-blue-600 border-b-2 border-blue-600' : 'hover:text-slat
          >
            Learn
          </span>
        </div>
      </div>

      <div className="flex items-center gap-6">
        <div className="flex bg-slate-100 p-1 rounded-xl border border-slate-200">
          <button className="px-5 py-1.5 text-[10px] font-bold bg-white text-blue-600 rounded-lg shadow-sm transition-all border border-slate-200">STOCKS</button>
          <button className="px-5 py-1.5 text-[10px] font-bold text-slate-500 hover:text-slate-900 transition-all">CRYPTO</button>
        </div>
        <div className="relative">
          <button
            onClick={() => setShowUserMenu(!showUserMenu)}
```

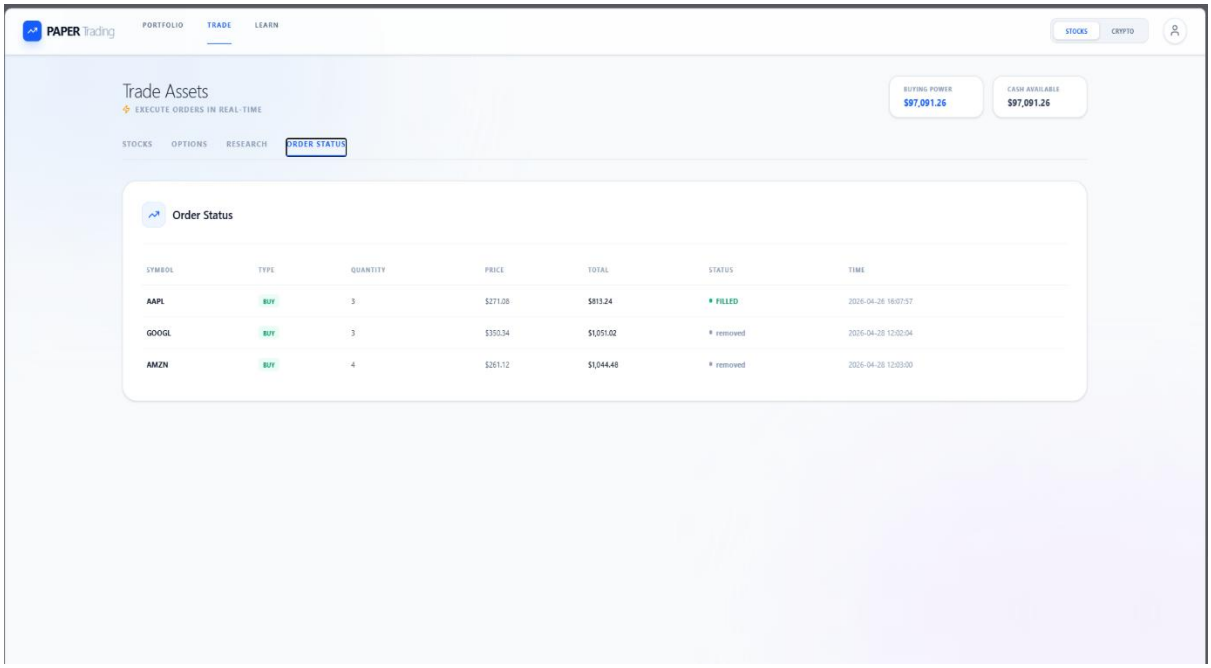
Project Snippets

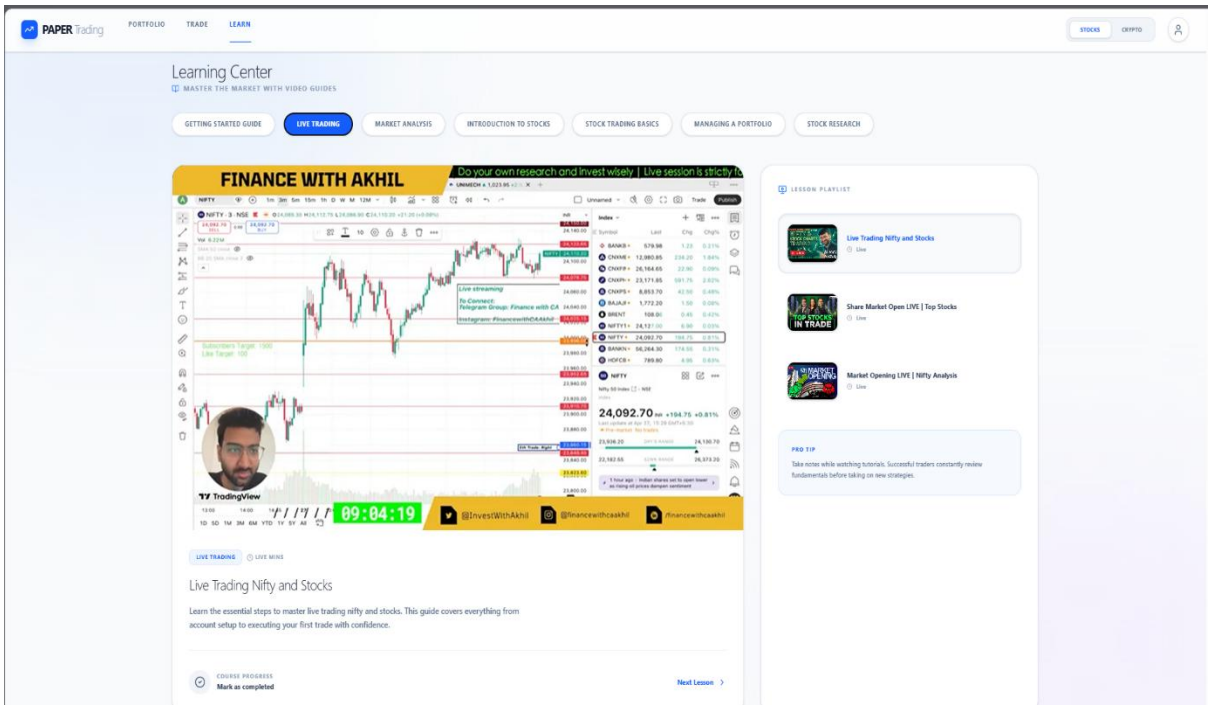





PAPER Trading
PORTFOLIO
TRADE
LEARN
STOCKS
CRYPTO
SYMBOL SEARCH
TSLA
TSLA
TESLA INC
LAST PRICE
$378.67 +2.37 (0.6298%)
DAY HIGH
$386.24
DAY LOW
$371.10
Tesla, Inc. · 1D · Cboe One
Indicators
Track all markets on TradingView
Options Chain EXPERIMENTAL
MAY 15, 2026
CALLS
STRIKE
PUTS


PAPER Trading
PORTFOLIO
TRADE
LEARN
STOCKS
CRYPTO
Trade Assets
EXECUTE ORDERS IN REAL-TIME
BUYING POWER
$99,186.76
CASH AVAILABLE
$99,186.76
STOCKS
OPTIONS
RESEARCH
ORDER STATUS
ASSET RESEARCH
GOOGL
GOOGL
NASDAQ STOCK MARKET
Alphabet Inc
$350.34
+5.94 (1.7247%)
REAL-TIME DATA · MARKET OPEN
Alphabet Inc (Google) Class A · 1D · Cboe One
Indicators
Track all markets on TradingView
EARNINGS
May 21
EPS
4.93
MARKET CAP
5.06 T
DIV YIELD
0.02%
P/E RATIO
40.73
VOLUME (CURRENT)
214.134m
52 WEEK HIGH ($)
212.19
DAY'S HIGH ($)
210.95
BID/ASK PRICE ($)
208.15 / 208.19
DAY'S LOW ($)
199.81
52 WEEK LOW ($)
104.08


PAPER Trading
PORTFOLIO
TRADE
LEARN
STOCKS
CRYPTO
Trade Assets
EXECUTE ORDERS IN REAL-TIME
BUYING POWER
$97,091.26
CASH AVAILABLE
$97,091.26
STOCKS
OPTIONS
RESEARCH
ORDER STATUS
Order Status
SYMBOL
TYPE
QUANTITY
PRICE
TOTAL
STATUS
TIME
AAPL
BUY
3
$271.08
$813.24
FILLED
2026-04-26 16:07:57
GOOGL
BUY
3
$350.34
$1,051.02
removed
2026-04-28 12:02:04
AMZN
BUY
4
$261.12
$1,044.48
removed
2026-04-28 12:03:00


PAPER Trading
PORTFOLIO
TRADE
LEARN
STOCKS
CRYPTO
Learning Center
MASTER THE MARKET WITH VIDEO GUIDES
GETTING STARTED GUIDE
LIVE TRADING
MARKET ANALYSIS
INTRODUCTION TO STOCKS
STOCK TRADING BASICS
MANAGING A PORTFOLIO
STOCK RESEARCH
FINANCE WITH AKHIL
Do your own research and invest wisely | Live session is strictly fo
24,092.70
09:04:19
@InvestWithAkhil
@financewithaakhil
/financewithaakhil
LIVE TRADING
Live Trading Nifty and Stocks
Learn the essential steps to master live trading nifty and stocks. This guide covers everything from account setup to executing your first trade with confidence.
COURSE PROGRESS
Mark as completed
Next Lesson
LESSON PLAYLIST
Live Trading Nifty and Stocks
Share Market Open LIVE | Top Stocks
Market Opening LIVE | Nifty Analysis
PRO TIP
Take notes while watching tutorials. Successful traders constantly review fundamentals before taking on new strategies.

Future Enhancement

- **AI-Based Recommendation**
  1. Smart stock suggestions using AI
  2. Risk analysis and prediction models
- **Advanced Data Visualization**
  1. Smart stock suggestions using AI
  2. Risk analysis and prediction models
- **Social Trading Features**
  1. Smart stock suggestions using AI
  2. Risk analysis and prediction models

GitHub Url

Athish-frontend/Paper-Trading-

Conclusion

The *Web-Based Paper Trading System for Stock Market Learning* is developed to provide a practical and risk-free platform for users to understand and experience stock market operations. The system successfully integrates learning, research, and trading simulation into a single unified platform, enabling users to bridge the gap between theoretical knowledge and real-world application. By using virtual money for trading, the platform allows beginners to practice and experiment without any financial risk, thereby improving their confidence and decision-making skills.

From a technical perspective, the system is implemented using modern technologies such as React.js for the frontend, Python with Flask for the backend, MongoDB for data storage, and Finnhub API for real-time stock market data. The adoption of a layered and component-based architecture ensures scalability, maintainability, and efficient performance. The system provides a user-friendly interface, real-time data updates, and essential trading functionalities, making it effective for both learning and simulation purposes.